Week 5

- UML
- Design Principles

UML - Unified Modified Language

- A common design language used to describe the structure and the behavior of software systems.

- UML provides a set of diagrams that help in designing and communicating software systems.

- Graphical notation used to communicate design of software systems
  - Describe the objects that form the system

UML - Unified Modified Language

- A common design language used to describe the structure and the behavior of software systems.

- UML provides a set of diagrams that help in designing and communicating software systems.

- Graphical notation used to communicate design of software systems
  - Describe the objects that form the system

Modelling Notations

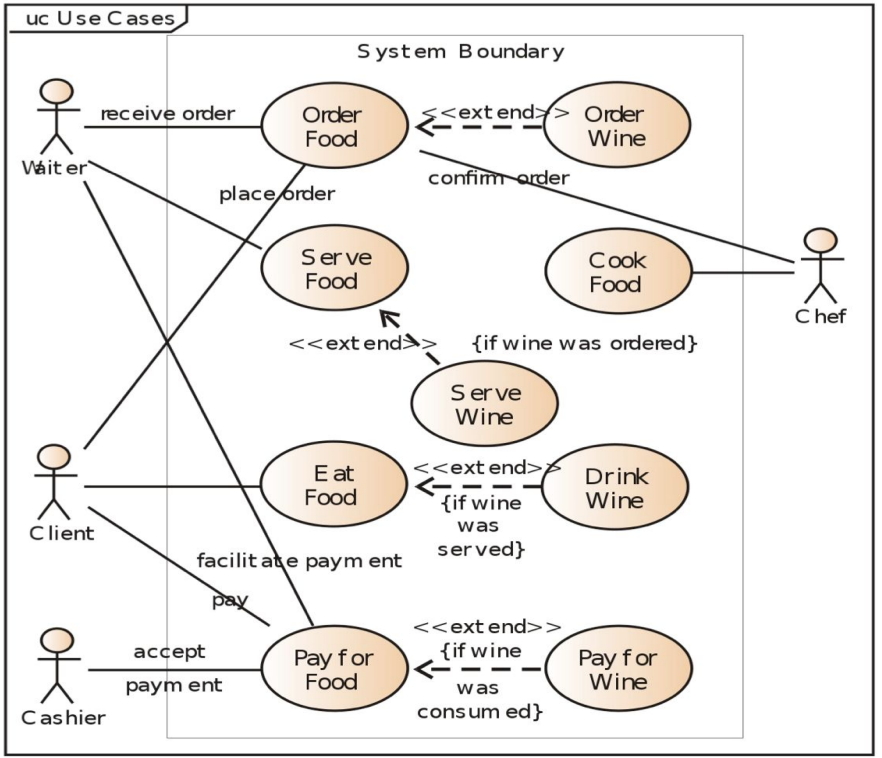
- Used for both requirements analysis and for specification and design • Useful for technical people
- Provide a high-level view
- Descendent of Entity-Relationship Diagram
- Describes data and operations
- Require training
- Many notations
    - each good for something
    - none good for everything

Usage of UML

- Help Developers communicate
- Provide Documentation
- Help find errors (tools check for consistency)
- Generate code (with tools)
- Drawing tools: Argo UML , Visio, Draw.io etc

Use Case Diagram

Actor + Action

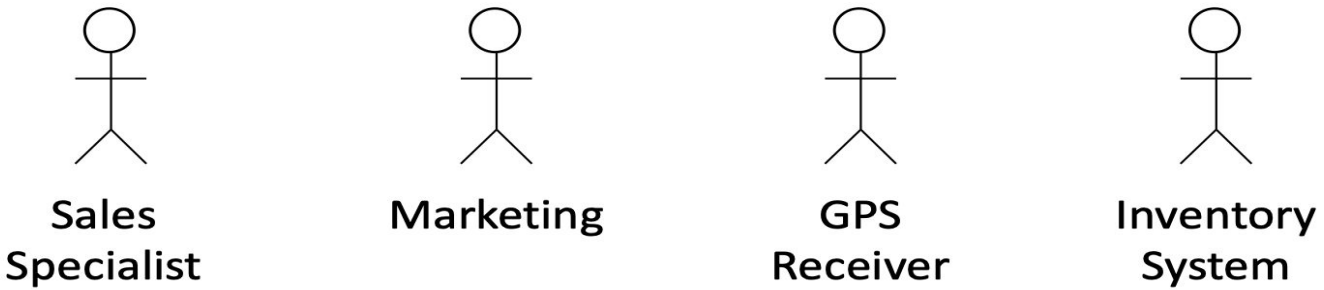


Defining Actors / Agents

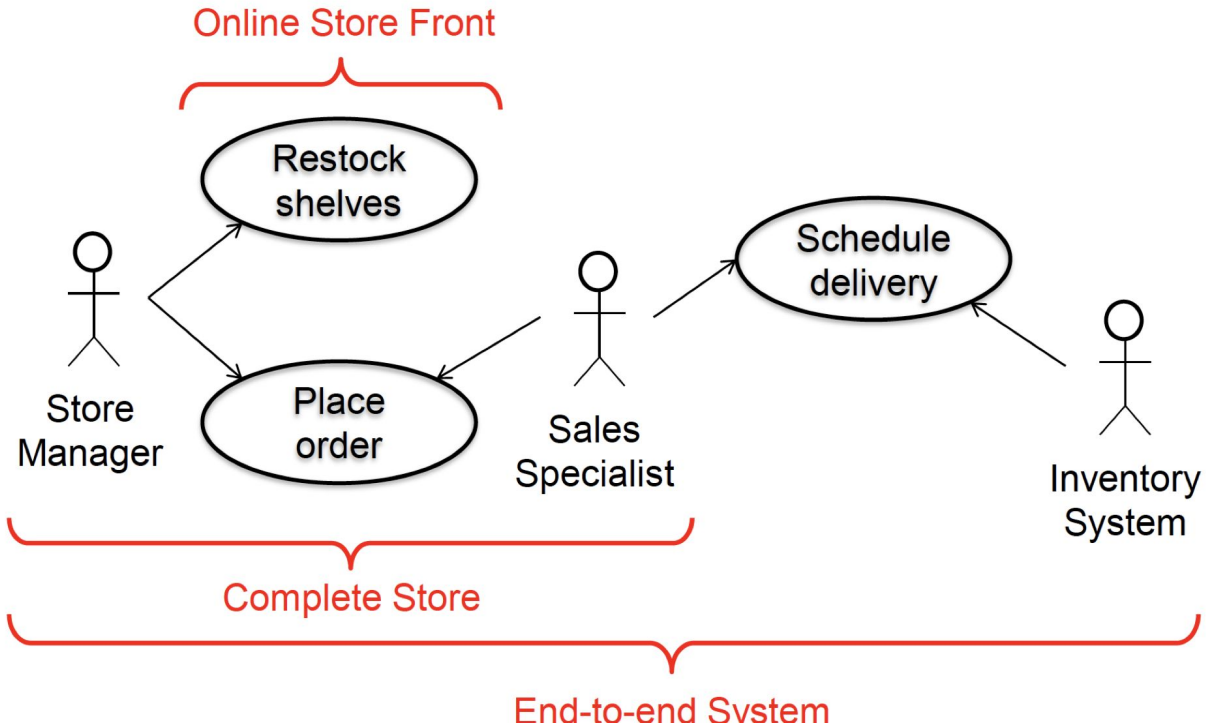
An actor is an entity that interacts with the system for the purpose of completing an event [Jacobson, 1992]

- Not as broad as stakeholders.

Actors can be a user, an organization, a device, or an external system.



Defining system Boundary

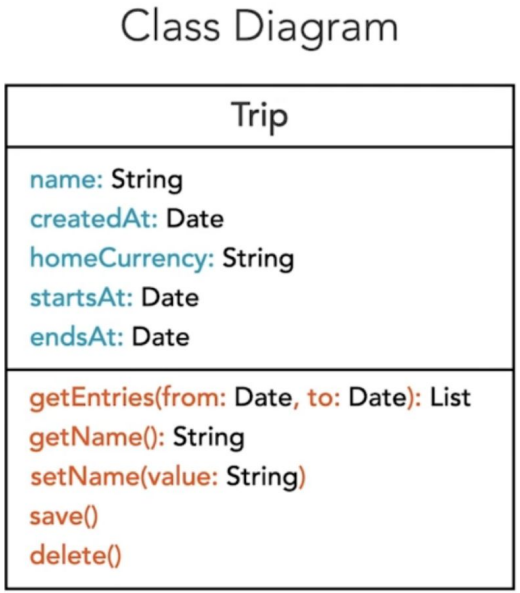


Use Case Diagram - Challenge

- Provides an overview of elevator system
- Try answering below questions:
- What actors interact with elevator system
- What are the main function of elevator system
- Take into account that elevators require maintenance and support 24 x 7

Class Diagram

- It’s used to visualize the structure of a software system.
- Class is represented by a rectangle with the name of the class in it.
- UML Class Diagrams consists of three sections: the c**lass name, attributes, and operations.**
- List the class attributes in a separate compartment. The attribute’s name and data type are separated by a colon.
- Method parameters appear within the parenthesis as name-data type pairs. If a method has a return value, add a colon after the closing parenthesis followed by the return type.



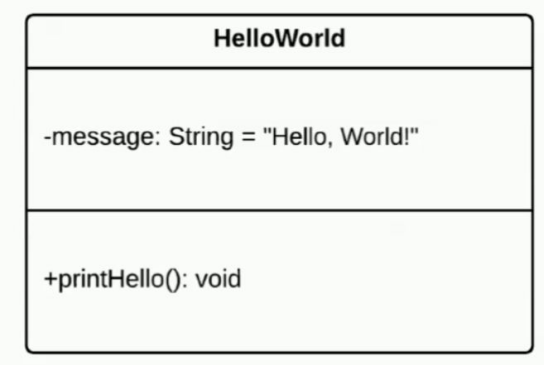
Attributes & Operations

Attributes represent the field or properties of the class. In UML , we express them as below

<visibility><name>:<type> = <default value><{modifier}>

Operations represent the method and behaviour of the class

<visibilit><name> (<parameterlist>) : <return type>

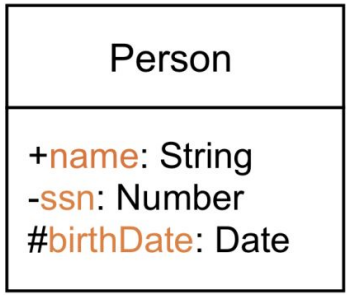


Class Visibility Levels

Public : A class method or attribute marked as public can be used by code outside of the object. It's represented by symbol "**+**".

Private: Private attributes and methods can only be used within the class that defines them. It's represented by symbol "**-**".

Protected: Only child classes and the defining class can access protected attributes or methods.It's represented by symbol "**#**".

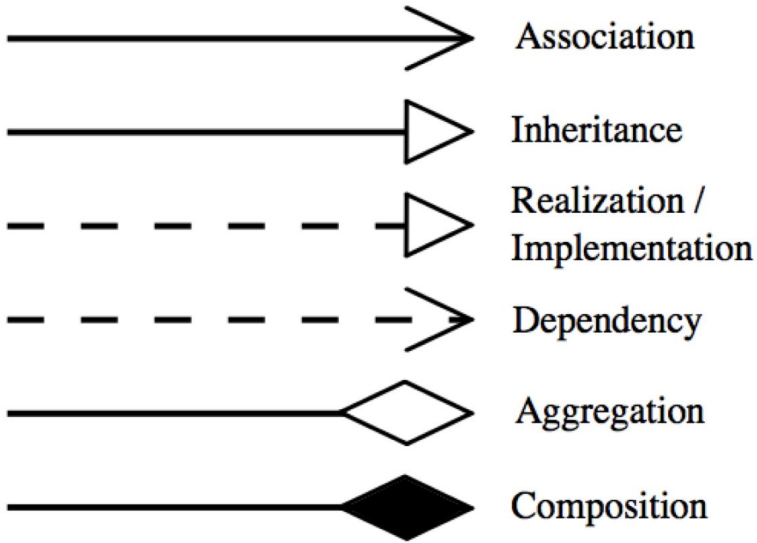


Relationships

Object oriented classes have relationship between them - they extend each other (***inheritance***), they depend on each other (***dependency***), they interact with each other (***association***) or they for part of each other (***aggregation or composition***)

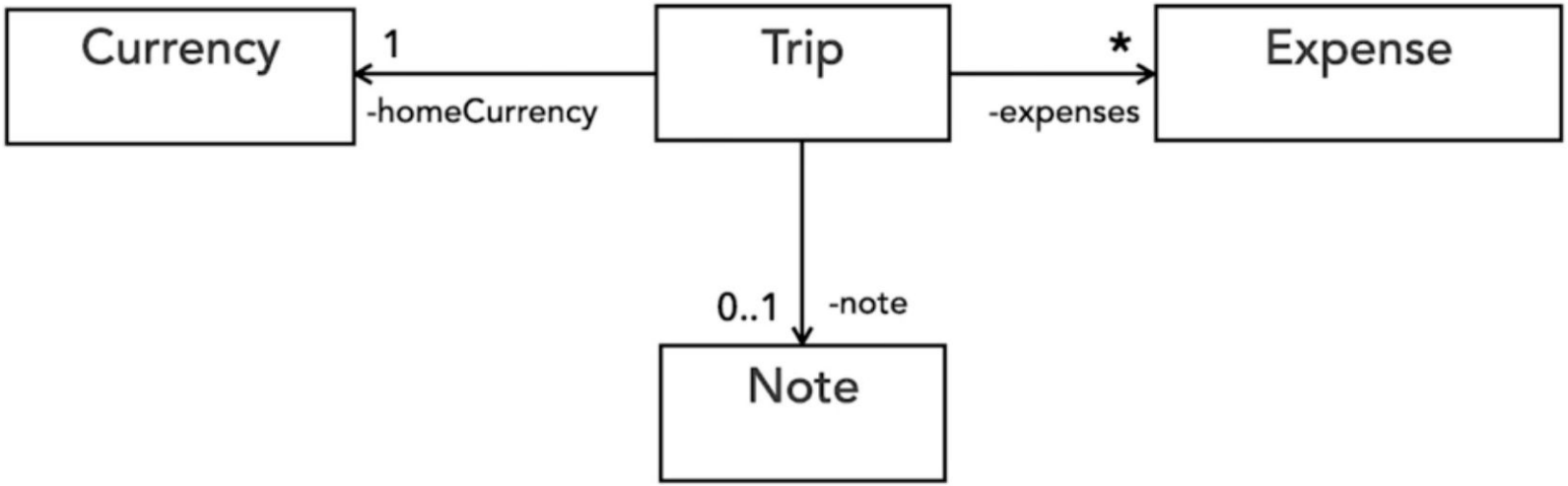
Relationships can also have ***multiplicity*** , which indicates how many instances of class exists (or can exist) on each side of the relationship

Relationship

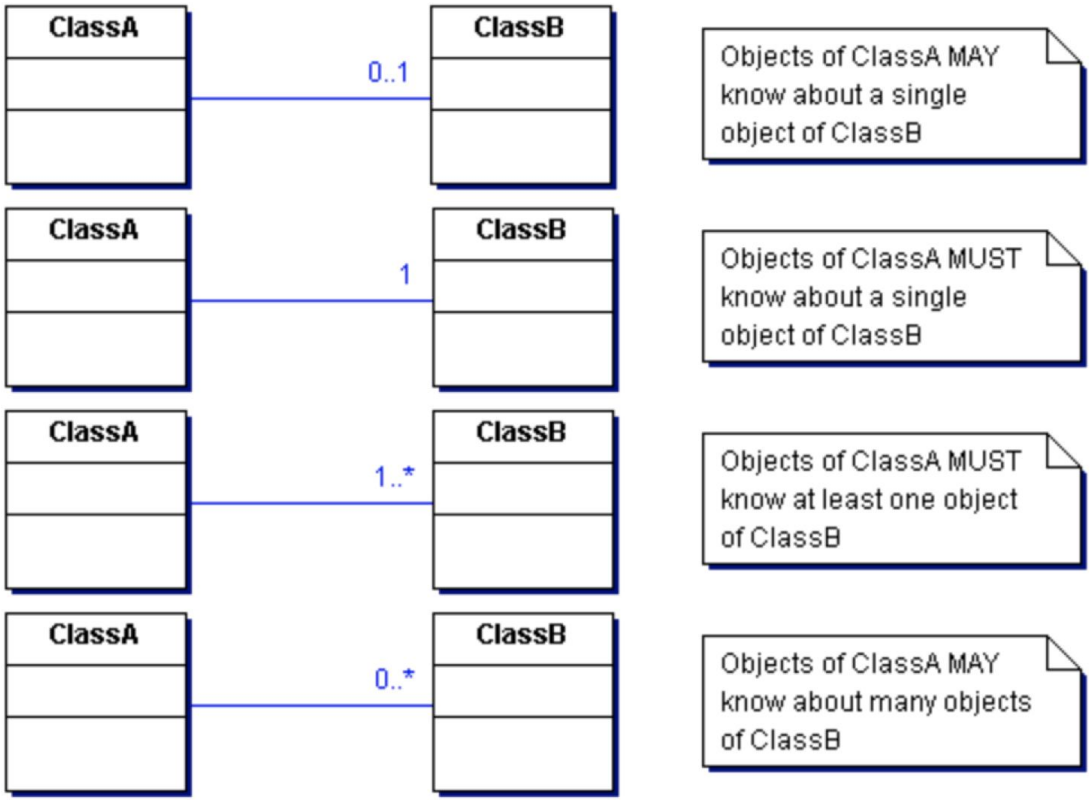


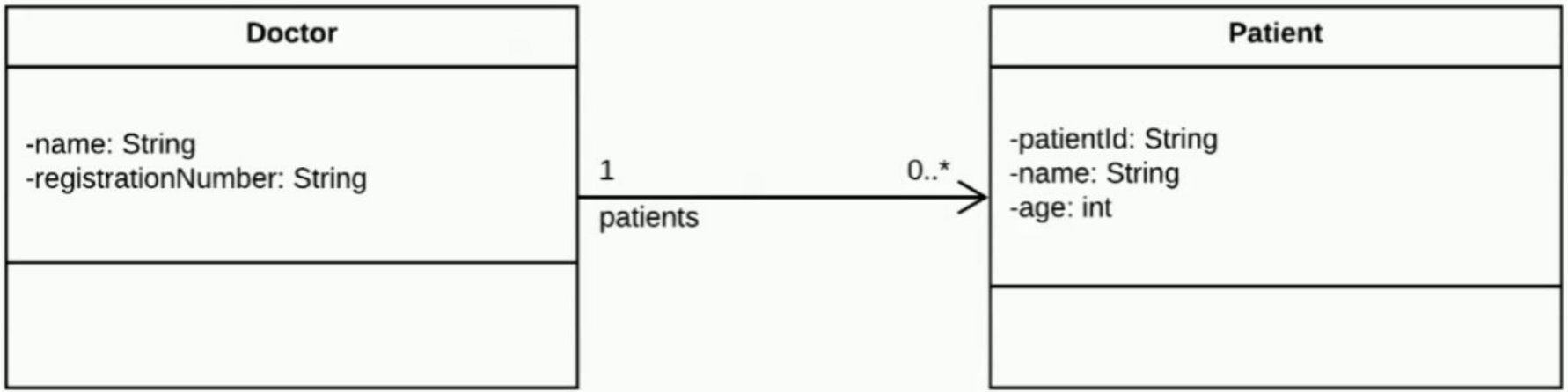
Association

Association “***interaction***” Used to visualize that the elements interact with/refer to each other.



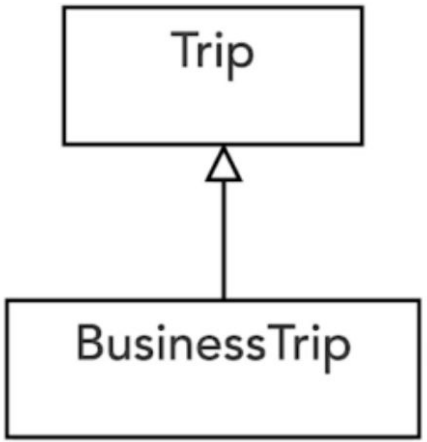
Contd.


ClassA
ClassB
0..1
Objects of ClassA MAY know about a single object of ClassB
ClassA
ClassB
1
Objects of ClassA MUST know about a single object of ClassB
ClassA
ClassB
1..*
Objects of ClassA MUST know at least one object of ClassB
ClassA
ClassB
0..*
Objects of ClassA MAY know about many objects of ClassB

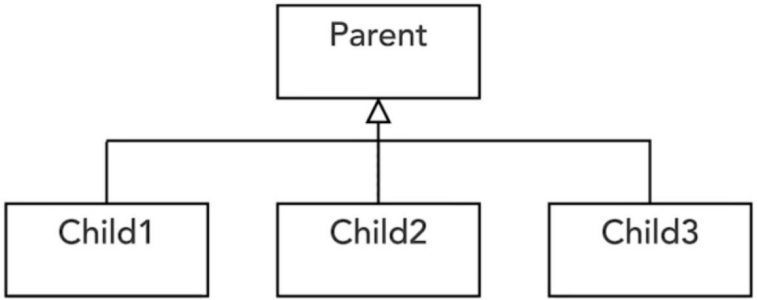

Doctor
-name: String
-registrationNumber: String
1
patients
0..*
Patient
-patientId: String
-name: String
-age: int

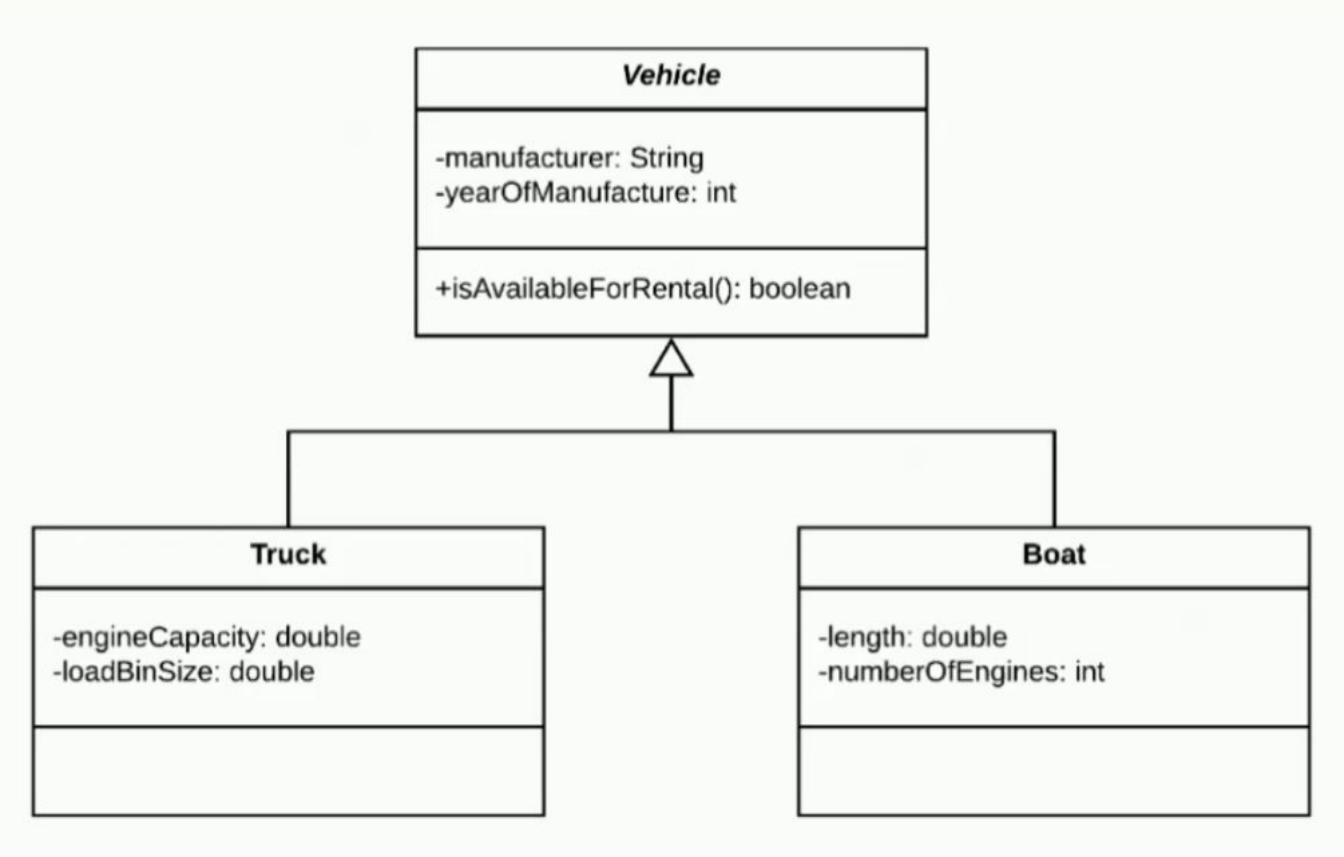
Generalization

Generalization “***is-a***” Represents that one element is based on another element.

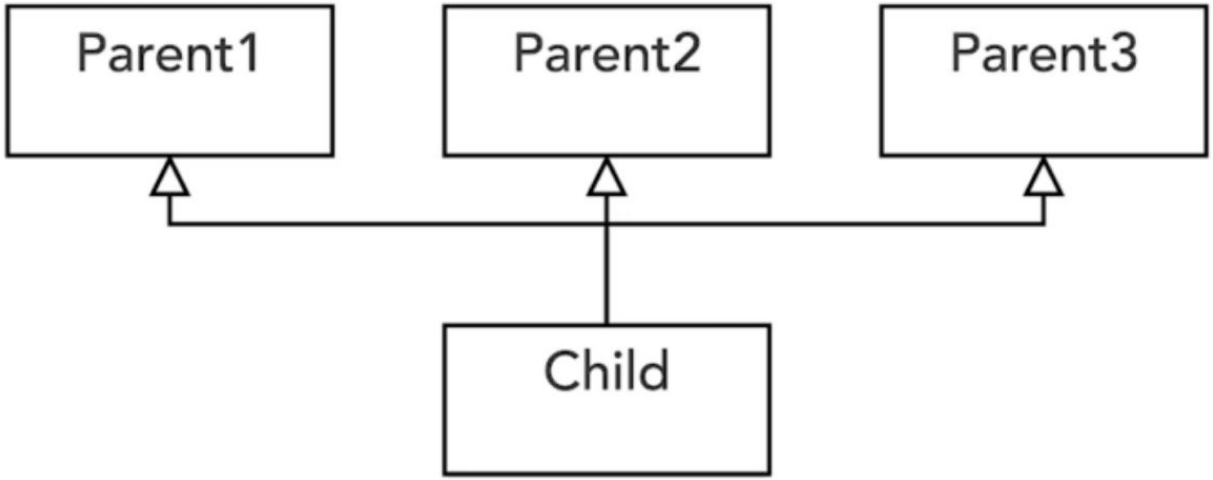


Parent/Child



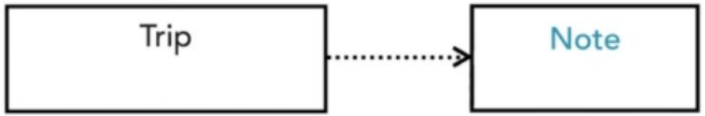

Vehicle
-manufacturer: String
-yearOfManufacture: int
+isAvailableForRental(): boolean
Truck
-engineCapacity: double
-loadBinSize: double
Boat
-length: double
-numberOfEngines: int

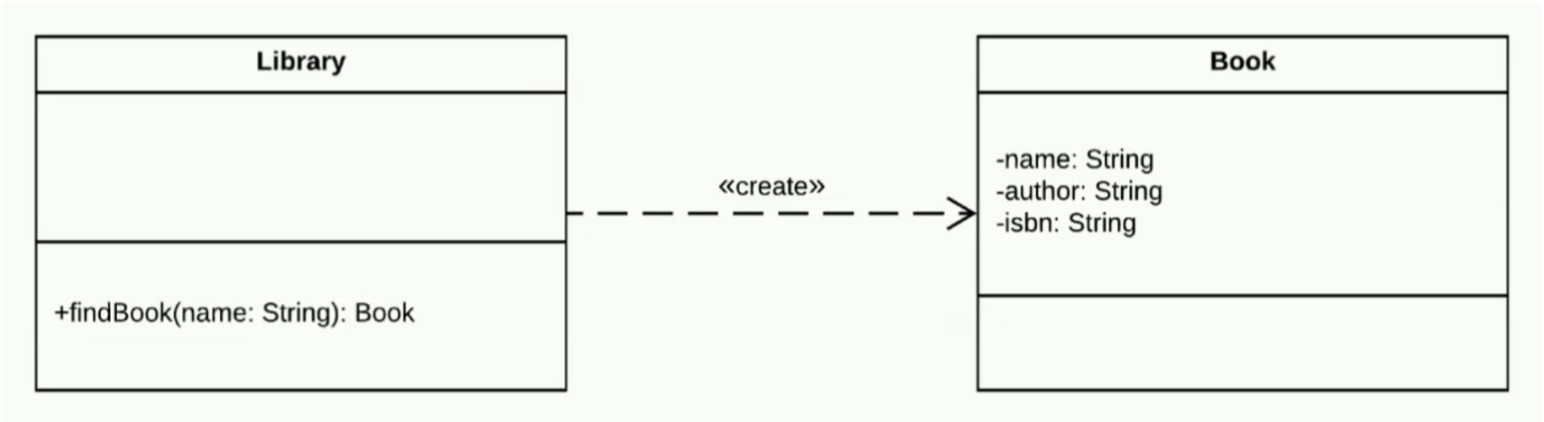
Multiple Inheritance



Dependency

Dependency “references” Indicates that one element receives a reference to another element (e.g. via a method parameter).

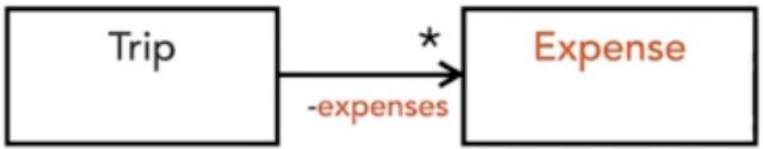




```
public class Library {

   public Book findBook(String name) {
      //Do some book stuff.
      return new Book();
   }
}
```

```
public class Book {

   private String name;
   private String author;
   private String isbn;

   //Getters/setters omitted.
}
```

Association vs Dependency

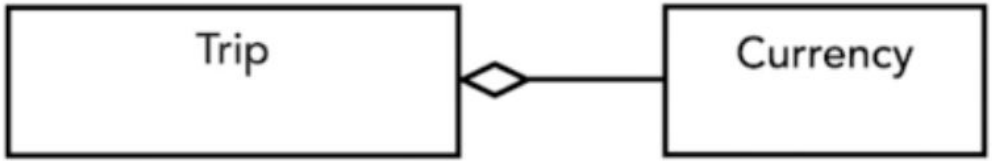


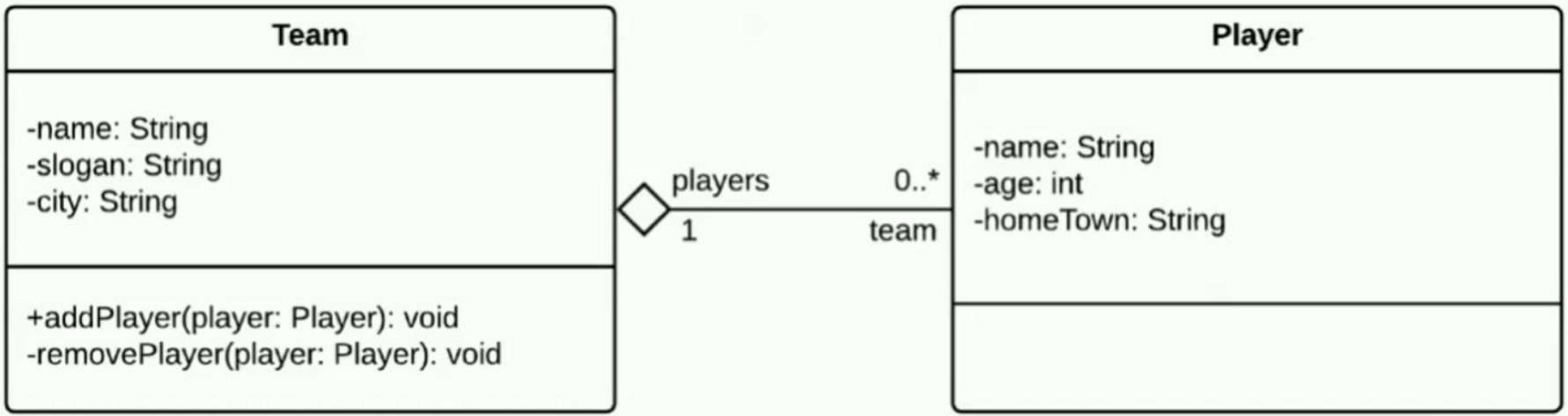
association

dependency

Aggregation

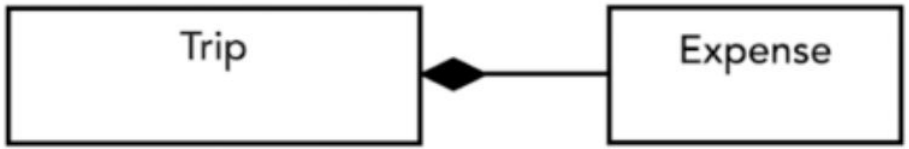
- “***has-a***”
- Aggregation represents a part-whole relationship and it's drawn as a solid line with a hollow diamond at the owner's end.
- This relationship is considered redundant because it expresses the same thing as the association.



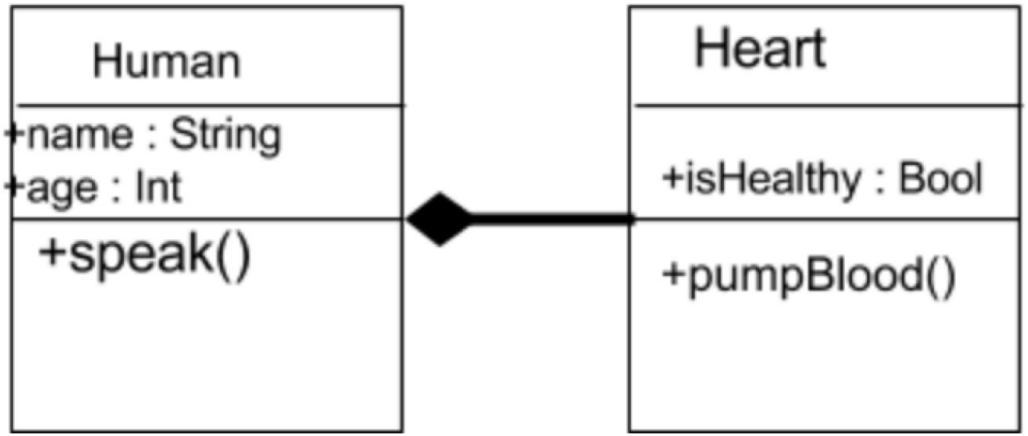

Team
-name: String
-slogan: String
-city: String
+addPlayer(player: Player): void
-removePlayer(player: Player): void
players
1
0..*
team
Player
-name: String
-age: int
-homeTown: String

Composition

- “***part-of***”
- Composition is a stronger form of association.
- It shows that the parts live and die with the whole.
- composition = ownership

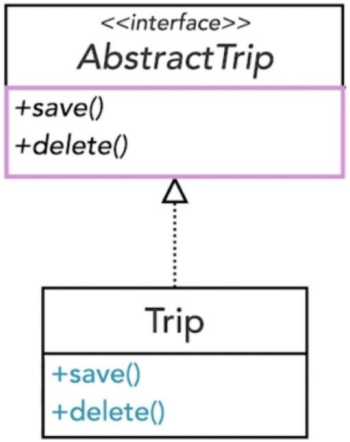


Contd.


Human
+name : String
+age : Int
+speak()
Heart
+isHealthy : Bool
+pumpBlood()

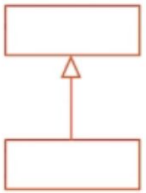
Realization

- A class ***implements the behavior*** specified by another model element.
- Represented as a hollow triangle on the interface end connected with dashed lines to the implementer classes.

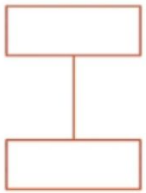


UML Relationships in Nutshell

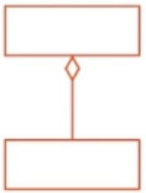
Generalization /
"is-a"



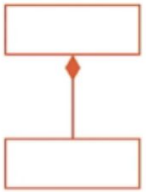
Association /
"has-a"



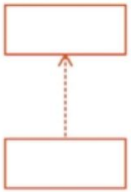
Aggregation /
"has-a"



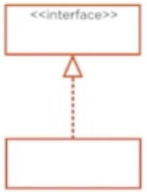
Composition /
"part-of"



Dependency /
"references"

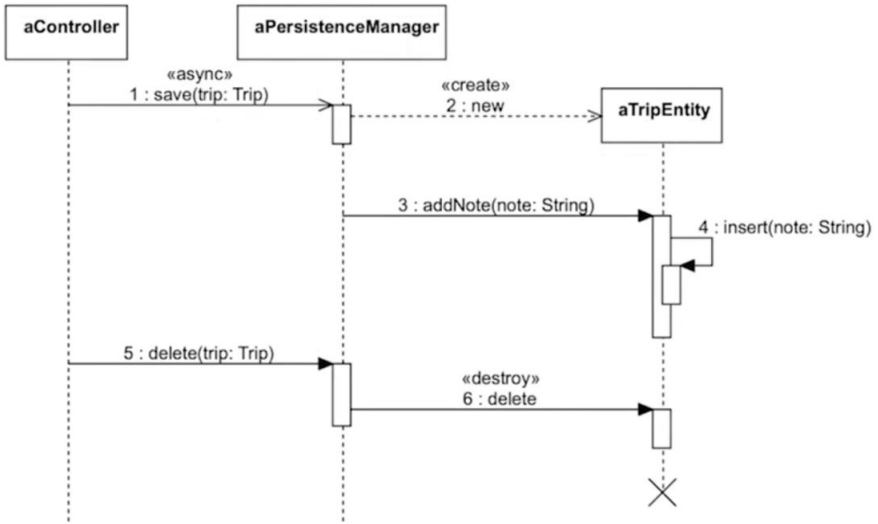


Realization /
"implements behavior"



Sequence Diagram

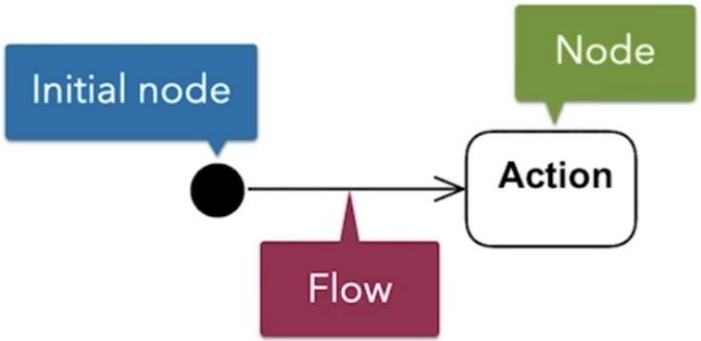
- Most common dynamic diagram is sequence diagram
- Describe the flow of logic in one particular scenario
- The time sequence of the objects participating in the interaction

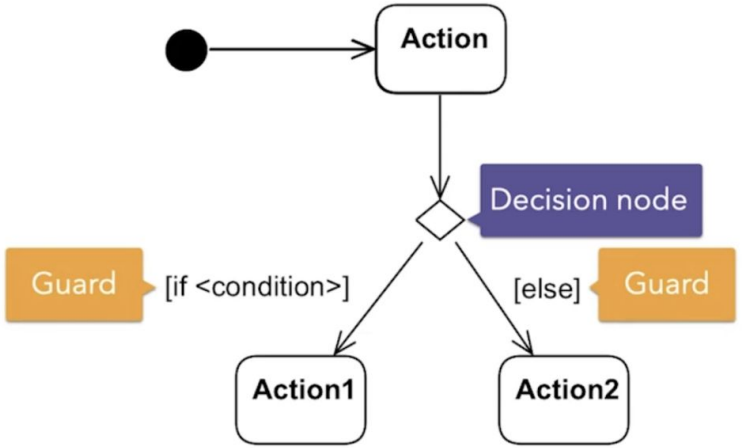


Activity Diagram

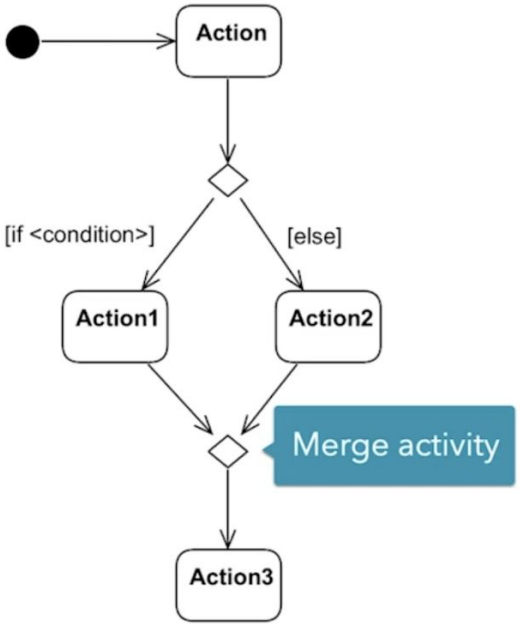
- Behavioral diagram to describe workflows or business processes
- Two main components:
  - Nodes - represent actions
  - Flow (edges) - represent flow of control between actions

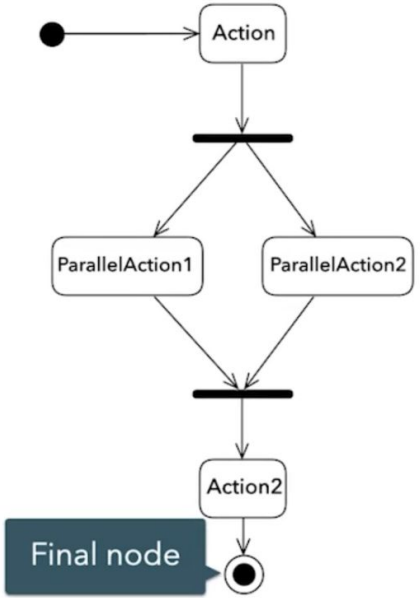
Action flow & Decision





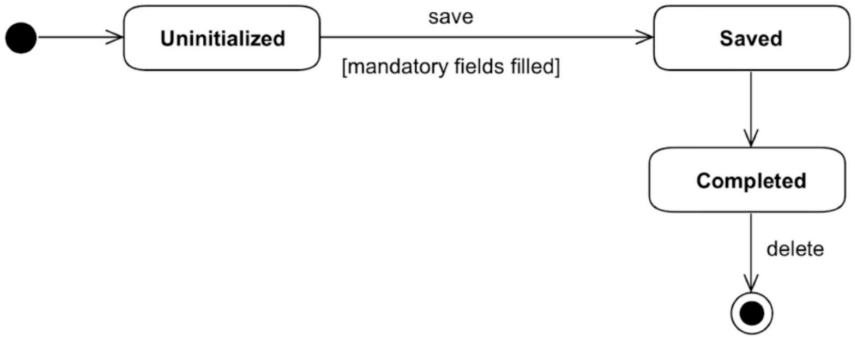
Merge and Concurrency





Statechart Diagram

- Describes the state changes of an object
- Main three components
  - State : represent the current condition of an object
  - Transition : Shows that an object switches state
  - Event : Triggers a transition



UML Example - Note taking App

- Clarify and collect examples
- Write User stories
- Map Use stories to use case diagrams
- Identify main entities and relationship
- Model Behavior
- Represent object states

Note Taking App

- Create and edit texts
- Attach documents, photos, and videos
- Capture hand drawn sketches
- Privacy - password protected
- Sync app data to cloud storage such as Dropbox, iCloud, or Google drive

Functional Requirements

- We need to build a note-taking app.
- Users can create and edit text-based nodes.
- A note may also include images or hand-drawn sketches.
- Notes should be password-protected for privacy.
- The app should automatically sync with cloud storage services to prevent data loss.

Non Functional Requirements

- The app should be easy to use and intuitive.
- The app must run on the latest version of iOS.
- The app should be able to handle a large number of notes without crashing.
- The app must be secure to protect the user's privacy.
- The app must include the support email and the link to the app's website.

Mapping Requirements to User Stories

Note Creation and Editing

- We need to build a note-taking app.
- Users can create and edit text-based nodes.
- A note may also include images or hand-drawn sketches.

Security

- Notes should be password-protected for privacy.

Cloud Sync

- The app should automatically sync with cloud storage services to prevent data loss

Contd.

Note Creation and Editing

- As a user, I want to create text-based notes to capture my thoughts.
- As a user, I want to edit my notes so that I can refine them over time.
- As a user, I want to include images or hand-drawn sketches in my notes so I can consolidate my memories.

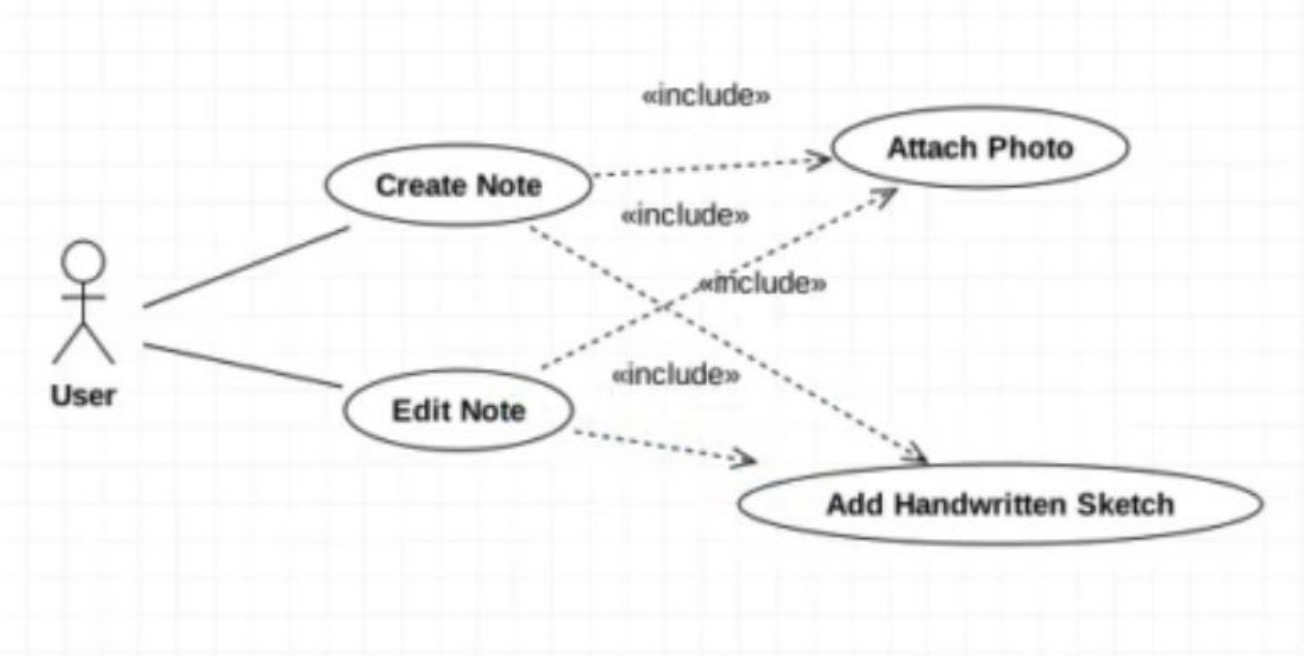
Security

- As a user, I want to secure sensitive notes with a password so that they remain secure and private.

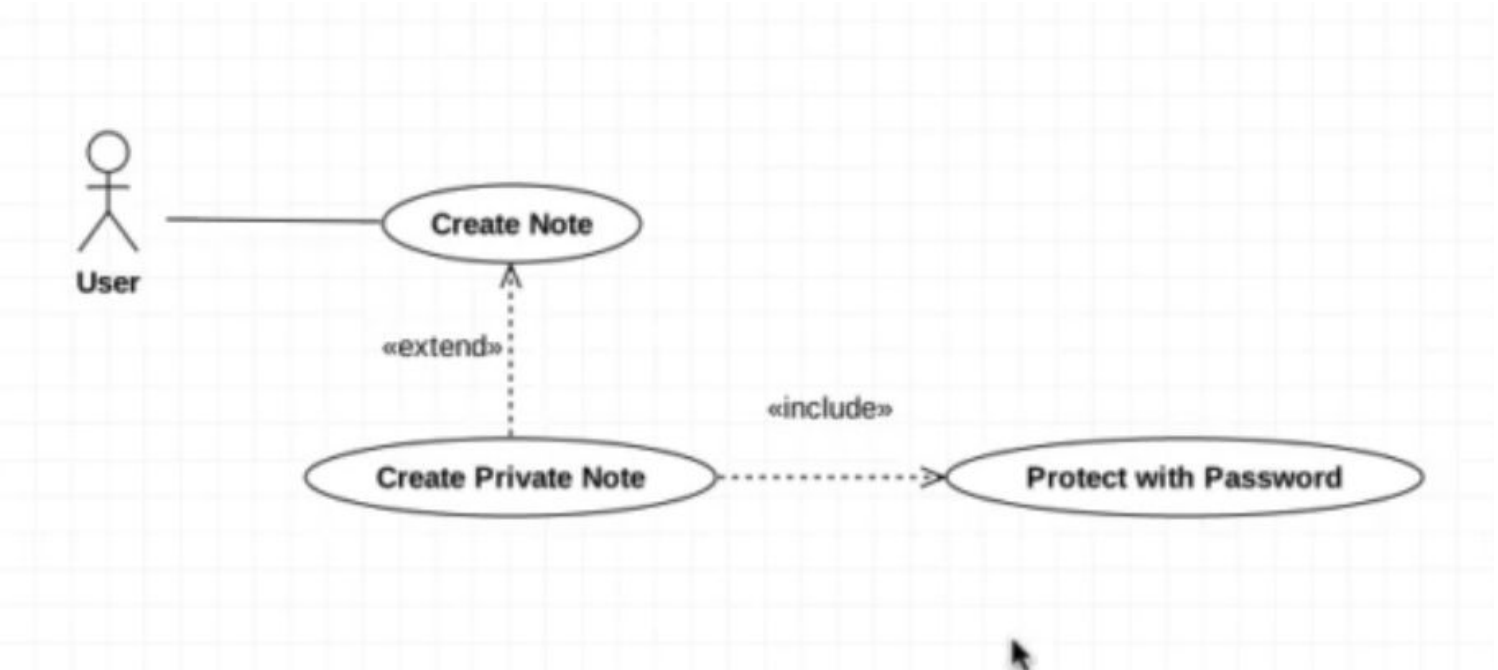
Cloud Sync

- As a user, I want my notes to automatically upload to cloud servers, so I always have a backup of my information.

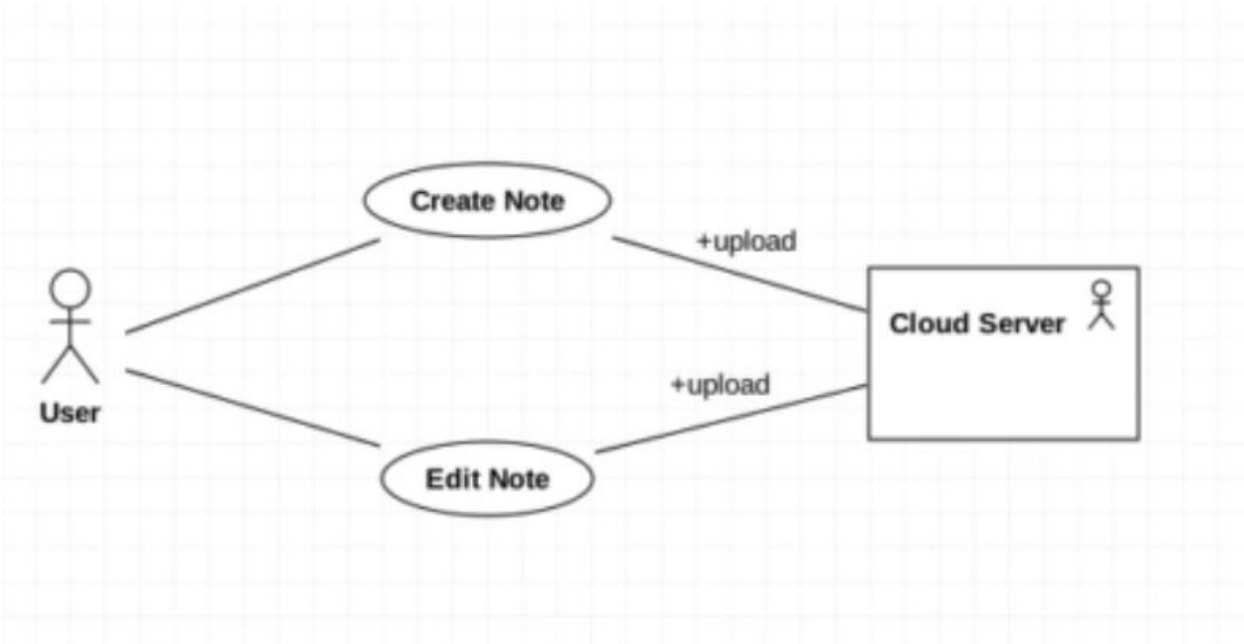
Diagramming Main Use Cases



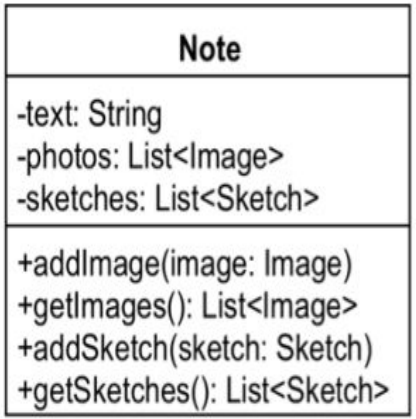
Contd.

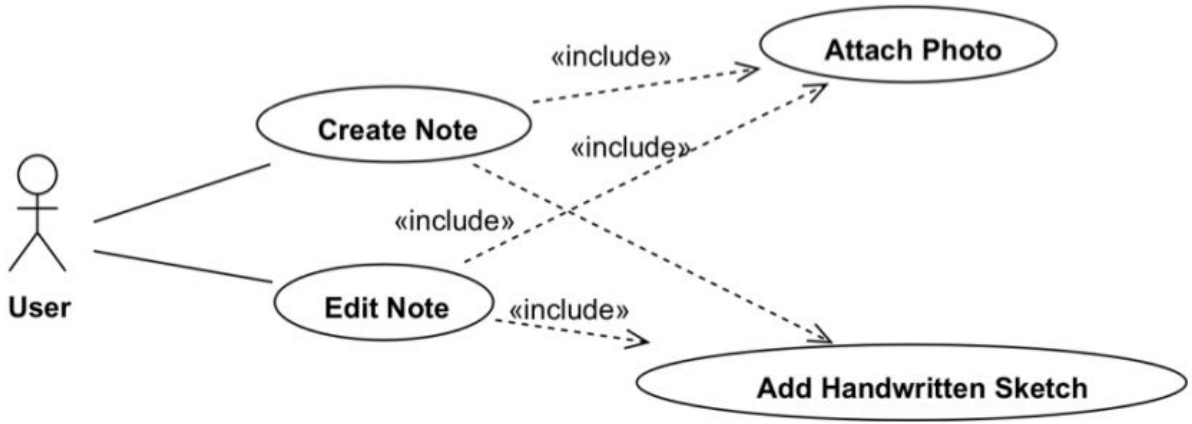

User
Create Note
«extend»
«include»
Create Private Note
Protect with Password

Contd.

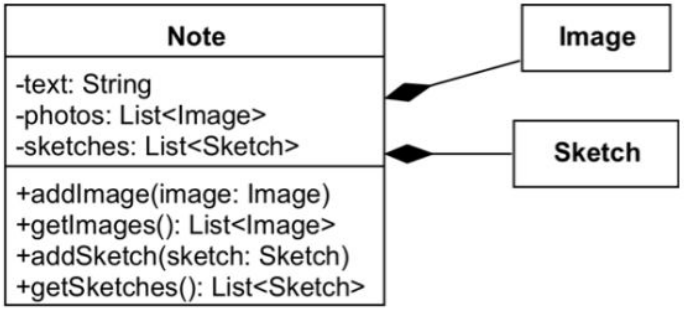


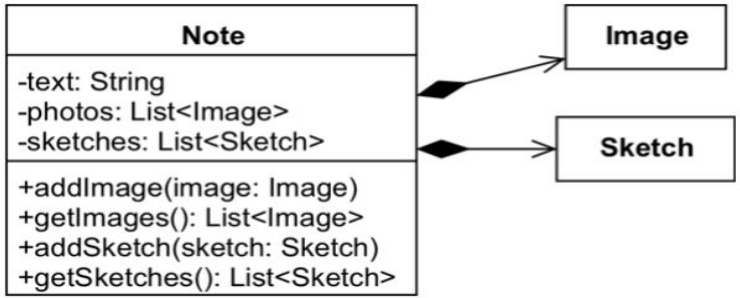
Modelling Classes and Relationship



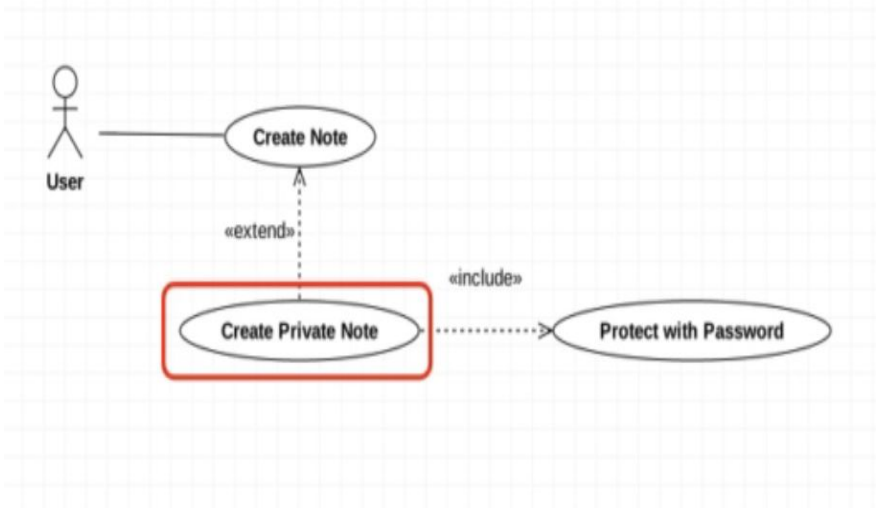


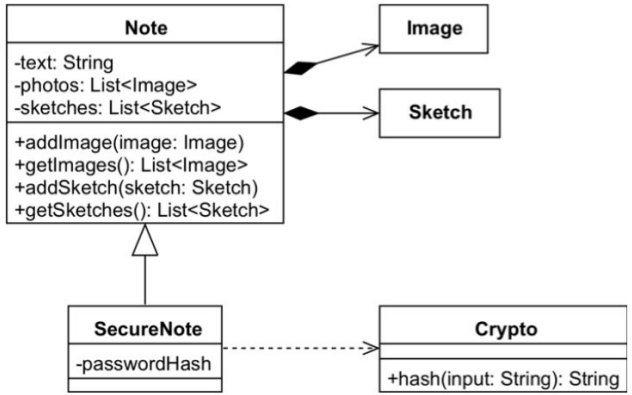
Contd.


Note
-text: String
-photos: List<Image>
-sketches: List<Sketch>
+addImage(image: Image)
+getImages(): List<Image>
+addSketch(sketch: Sketch)
+getSketches(): List<Sketch>
Image
Sketch

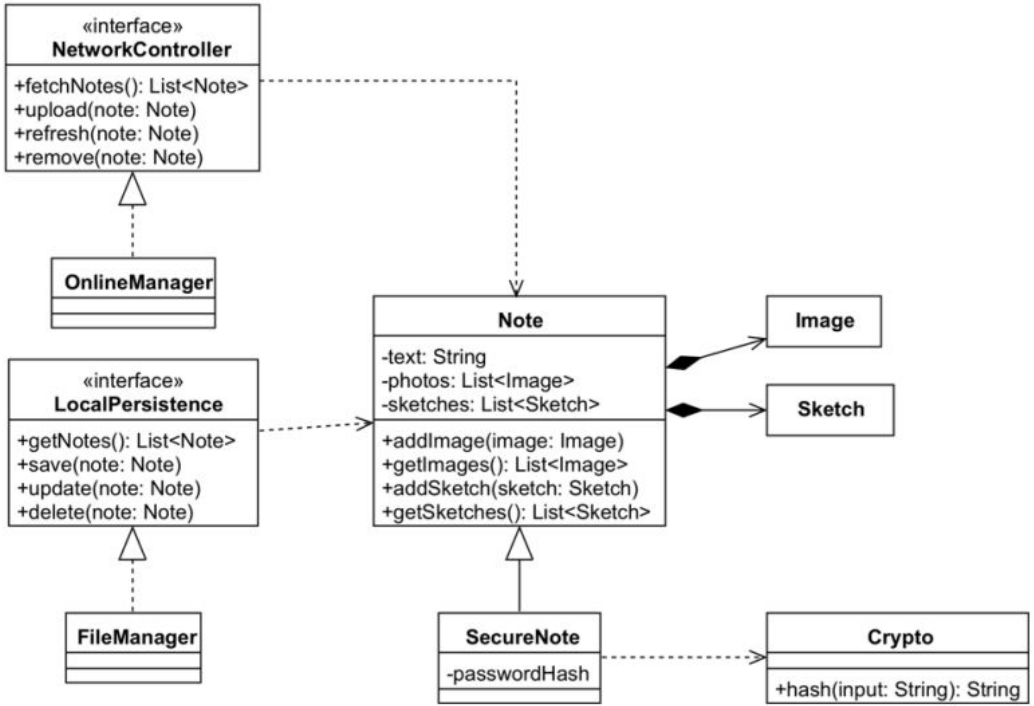

Note
-text: String
-photos: List<Image>
-sketches: List<Sketch>
+addImage(image: Image)
+getImages(): List<Image>
+addSketch(sketch: Sketch)
+getSketches(): List<Sketch>
Image
Sketch

Contd.

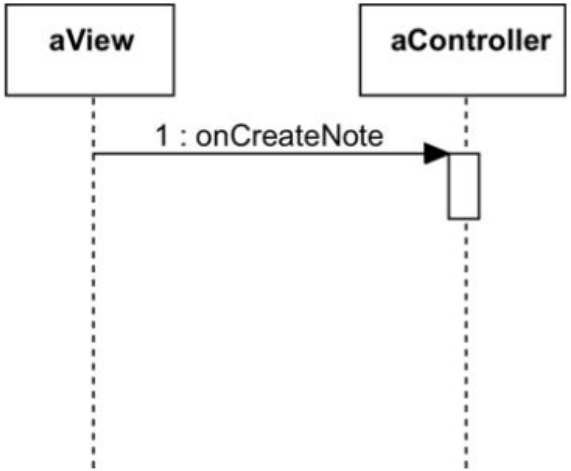

User
Create Note
«extend»
«include»
Create Private Note
Protect with Password

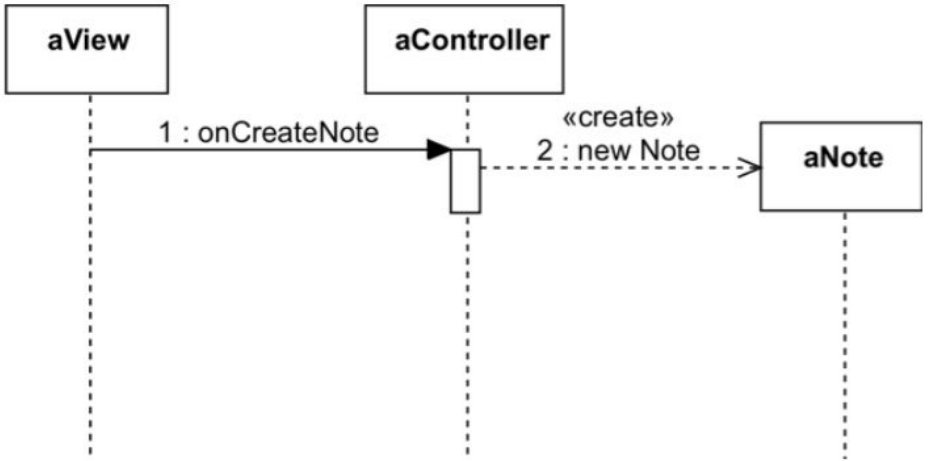

Note
-text: String
-photos: List<Image>
-sketches: List<Sketch>
+addImage(image: Image)
+getImages(): List<Image>
+addSketch(sketch: Sketch)
+getSketches(): List<Sketch>
Image
Sketch
SecureNote
-passwordHash
Crypto
+hash(input: String): String

Contd.

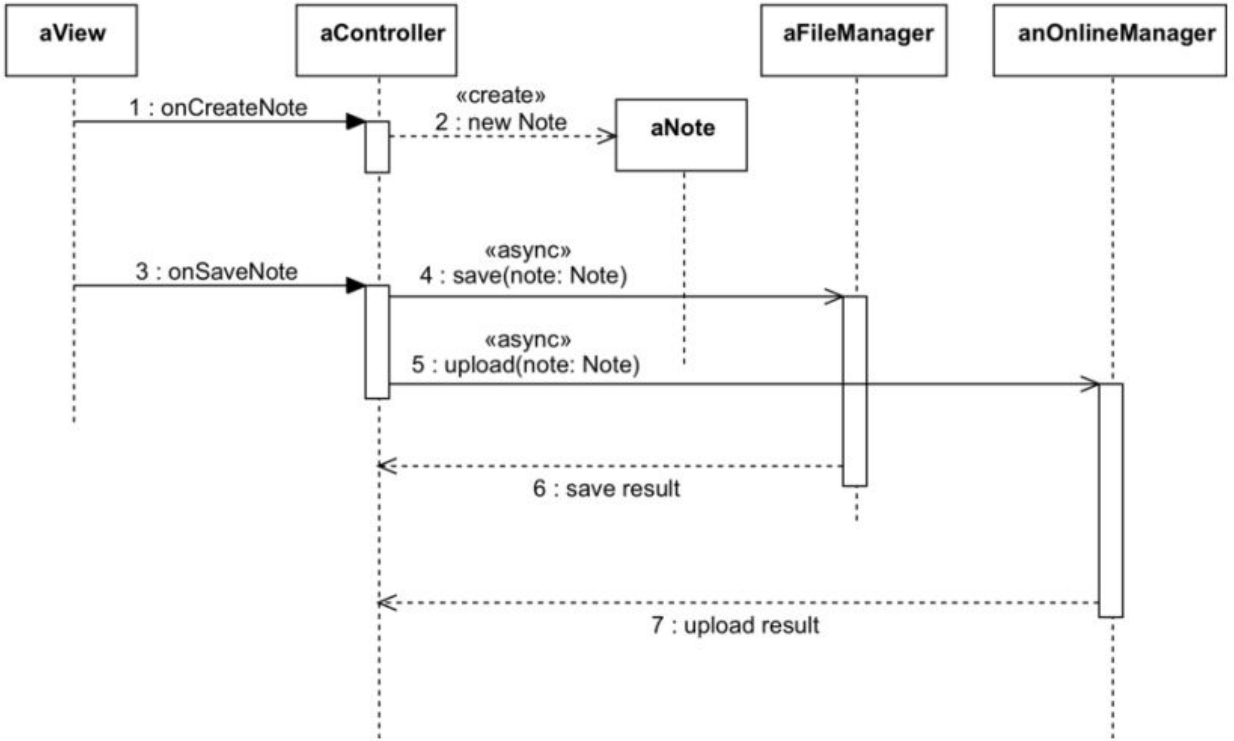

«interface»
NetworkController
+fetchNotes(): List<Note>
+upload(note: Note)
+refresh(note: Note)
+remove(note: Note)
OnlineManager
«interface»
LocalPersistence
+getNotes(): List<Note>
+save(note: Note)
+update(note: Note)
+delete(note: Note)
FileManager
Note
-text: String
-photos: List<Image>
-sketches: List<Sketch>
+addImage(image: Image)
+getImages(): List<Image>
+addSketch(sketch: Sketch)
+getSketches(): List<Sketch>
Image
Sketch
SecureNote
-passwordHash
Crypto
+hash(input: String): String

Creating Sequence Diagram

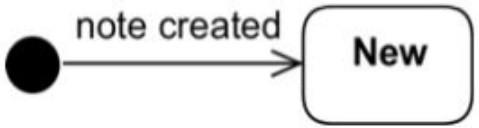


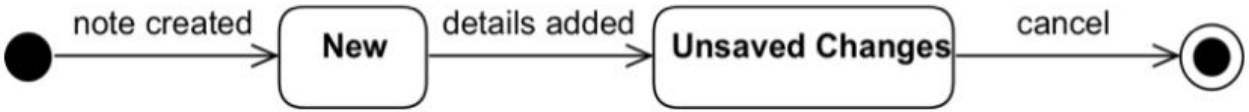


Contd.

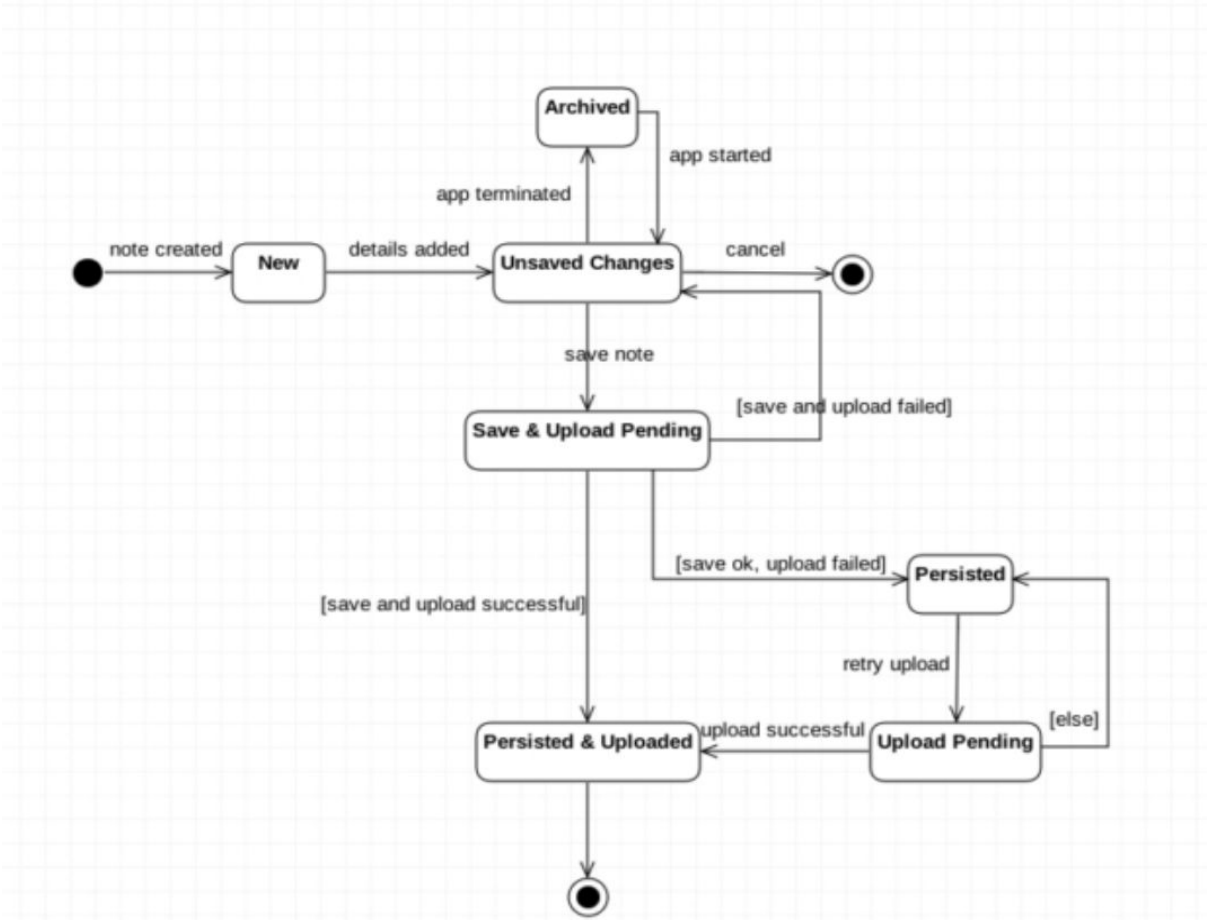

aView
aController
aFileManager
anOnlineManager
aNote
1 : onCreateNote
«create»
2 : new Note
3 : onSaveNote
«async»
4 : save(note: Note)
«async»
5 : upload(note: Note)
6 : save result
7 : upload result

Note Object State Chart Diagram

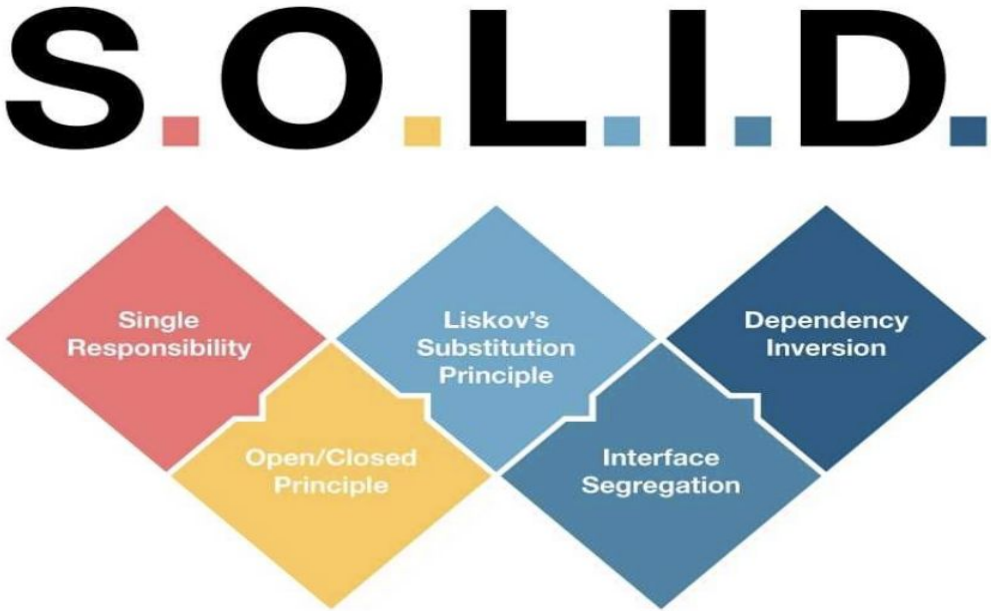




Contd.


Archived
app started
app terminated
note created
New
details added
Unsaved Changes
cancel
save note
[save and upload failed]
Save & Upload Pending
[save ok, upload failed]
Persisted
[save and upload successful]
retry upload
[else]
Persisted & Uploaded
upload successful
Upload Pending

OO Design Principles



Single Responsibility Principle

- Every Software component should have one and only one responsibility
  - Component can be class , method or module

- A class should have one, and only one, reason to change. Just because you can, doesn't mean you should

Single Responsibility Principle

Guildelines to partition your logic into classes

Contd.

Contd

Cohesion - In the software world. is defined as the degree to which the various parts of a software component are related.

In Single Responsibility Principle - we should always aim for **high cohesion** within a component,

If there is high cohesion between all methods of a class, we can assign a single responsibility to all the methods as a whole. For e.g (next page) - for the first class Square, we can safely say that the responsibility of the Square class a whole is to deal with the measurements related to a square.

Similary , for the second class SquareUI, we can safely say that the responsibility of the SquareUI class is to deal with rendering the image of a square.

```java
public class Square {

	int side = 5;

	public int calculateArea() {
		return side * side;
	}

	public int calculatePerimeter() {
		return side * 4;
	}

	public void draw() {
		if (highResolutionMonitor) {
		    // Render a high resolution image of a square
		} else {
		    // Render a normal image of a square
		}
	}

	public void rotate(int degree) {
		// Rotate the image of the square clockwise to
		// the required degree and re-render
	}

}
```

```java
public class Square {

	int side = 5;

	public int calculateArea() {
		return side * side;
	}

	public int calculatePerimeter() {
		return side * 4;
	}

}
```

```java
public class SquareUI {

	public void draw() {
		if (highResolutionMonitor) {
		    // Render a high resolution image of a square
		} else {
		    // Render a normal image of a square
		}
	}

	public void rotate(int degree) {
		// Rotate the image of the square clockwise to
		// the required degree and re-render
	}

}
```

Coupling

Coupling is defined as the level of inter dependency between various software components.

Two types of gauge shown in fig

- Standard Guage is 1.4 m wide.
- Broad guage is 1.6m wide.

```java
public class Student {

	private String studentId;
	private Date studentDOB;
	private String address;

	public void save() {
		// Serialize object into a string representation
		String objectStr = MyUtils.serializeIntoAString(this);
		Connection connection = null;
		Statement stmt = null;
		try {
		Class.forName("com.mysql.jdbc.Driver");
		connection =
		DriverManager.getConnection("jdbc:mysql://localhost:3306/
		MyDB", "root", "password");
		stmt = connection.createStatement();
		stmt.execute("INSERT INTO STUDENT VALUES (" + objectStr +
		")");
		} catch (Exception e) {
				e.printStackTrace();
		}
	}

	public String getStudentId() {
		return studentId;
	}

	public void setStudentId(String studentId) {
		this.studentId = studentId;
	}

	...
	...
	....
	......
}
```

```java
public class StudentRepository {
	public void save(Student student) {
		// Serialize object into a string representation
		String objectStr = MyUtils.serializeIntoAString(student);
		Connection connection = null;
		Statement stmt = null;
		try {
			Class.forName("com.mysql.jdbc.Driver");
			connection =
			DriverManager.getConnection("jdbc:mysql://localhost:
			3306/MyDB", "root", "password");
			stmt = connection.createStatement();
			stmt.execute("INSERT INTO STUDENT VALUES (" +
			objectStr + ")");
		} catch (Exception e) {
				e.printStackTrace();
		}
	}
}
```

```java
public class Student {

	private String studentId;
	private Date studentDOB;
	private String address;

	public void save() {
		new StudentRepository().save(this);
	}

	public String getStudentId() {
		return studentId;
	}

	public void setStudentId(String studentId) {
		this.studentId = studentId;
	}
```

Open Closed Principle

Software entities should be open for *extension*, but closed for *modification*.


Wii
Wii


Wii

Contd.

Closed for Modification

New features getting added to the software component,should NOT have to modify existing code

Open for Extension

A software component should be extendable to add a new feature or to add a new behaviour to it

```java
public class InsurancePremiumDiscountCalculator {

	public int
	calculatePremiumDiscountPercent(HealthInsuranceCustomerProfile customer)
	{
		if (customer.isLoyalCustomer()) {
			return 20;
		}
		return 0;
	}

}
```

```java
public class HealthInsuranceCustomerProfile {

	public boolean isLoyalCustomer() {
		return true;//or false
	}

}
```

```java
public class VehicleInsuranceCustomerProfile {

	public boolean isLoyalCustomer() {
		return true;//or false
	}

}
```

```
public class InsurancePremiumDiscountCalculator {

	public int calculatePremiumDiscountPercent(HealthInsuranceCustomerProfile customer) {
		if (customer.isLoyalCustomer()) {
			return 20;
		}
		return 0;
	}

	public int calculatePremiumDiscountPercent(VehicleInsuranceCustomerProfile customer)
	{
		if (customer.isLoyalCustomer()) {
			return 20;
		}
		return 0;
	}
}
```


public class InsurancePremiumDiscountCalculator {
public int
calculatePremiumDiscountPercent(CustomerProfile customer)
{
if (customer.isLoyalCustomer()) {
return 20;
}
return 0;
}
}
public interface CustomerProfile {
public boolean isLoyalCustomer();
}
public class
HealthInsuranceCustomerProfile
implements CustomerProfile {
@Override
public boolean isLoyalCustomer()
{
return true;// or false
public class
VehicleInsuranceCustomerProfile
implements CustomerProfile {
@Override
public boolean isLoyalCustomer()
{
return true;// or false

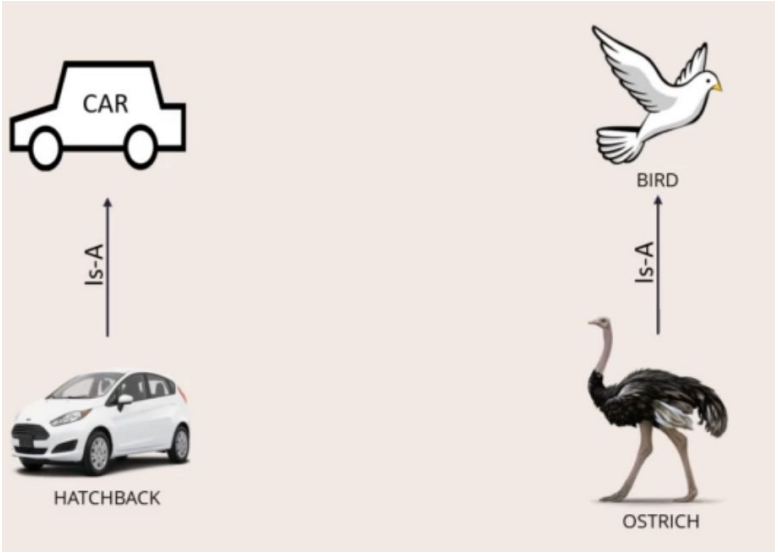
Advantages and Disadvantages

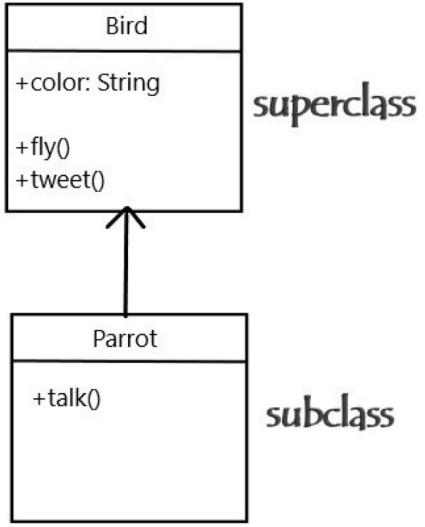
- Easy to add new features
- Leads to minimal cost of developing and testing software
- Open closed principle often requires decoupling , which in turn, automatically follows SRP
- SOLID principles are intertwined and interdependent
- SOLID principles are more effective when they are combined together

- Do not follow OCP blindly and revamping of design should be done only when repeated occurrence is seen

Liskov Substitution Principle

Objects should be replaceable with their subtypes without affecting the correctness of the program





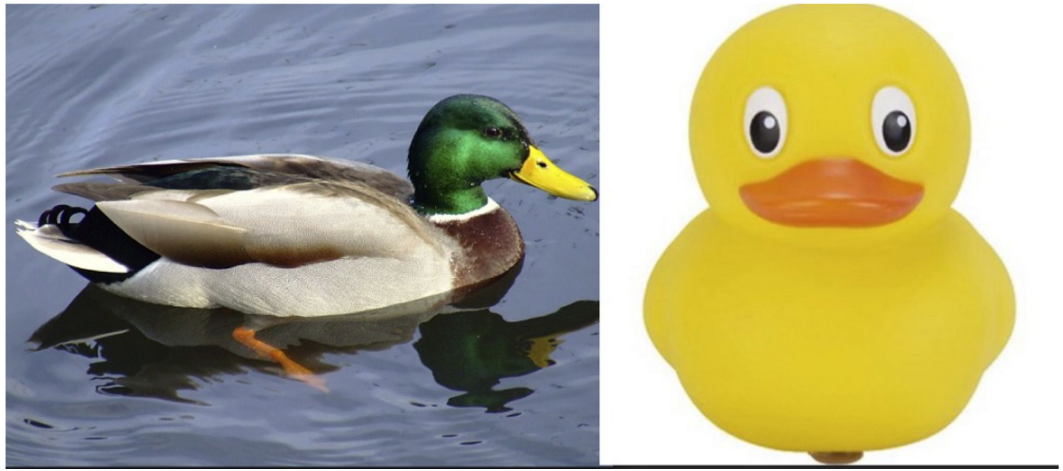
IS-A way of Thinking

```java
public class Bird {

    public void fly() {
        //Fly high!
    }

}
```

```java
public class Ostrich extends Bird {

    @Override
    public void fly() {
        // Unimplemented
        throw new RuntimeException();
    }

}
```

Change the way Is-A way of thinking

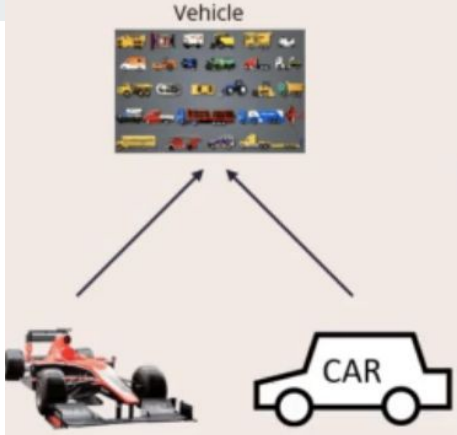
Duck Test - If it looks like a duck and quacks like a duck but it needs batteries , you probably have the wrong abstraction.



```java
public class Car {

	public double getCabinWidth() {
		//return cabin width
	}

}
```

```java
public class RacingCar extends Car {

	@Override
	public double getCabinWidth() {
		//UNIMPLEMENTED!
	}

	public double getCockpitWidth() {
		//return cockpit width
	}

}
```

```
public class CarUtils {

public static void main(String[] args) {
      Car first = new Car();
      Car second = new Car();
      Car third = new RacingCar();

      List<Car> myCars = new ArrayList<>();
      myCars.add(first);
      myCars.add(second);
      myCars.add(third);

      for(Car car: myCars) {
            System.out.println(car.getCabinWidth());
      }

      }

}
```




```java
public class Vehicle {

	public double getInteriorWidth() {
		//return interior width
	}

}
```

```java
public class Car extends Vehicle{

	@Override
	public double getInteriorWidth() {
		return this.getCabinWidth();
	}

	public double getCabinWidth() {
		//return cabin width
	}

}
```

```java
public class RacingCar extends Vehicle {

	@Override
	public double getInteriorWidth() {
		return this.getCockpitWidth();
	}

	public double getCockpitWidth() {
		//return cockpit width
	}

}
```

```java
public class VehicleUtils {

public static void main(String[] args) {
	Vehicle first = new Car();
	Vehicle second = new Car();
	Vehicle third = new RacingCar();

	List<Vehicle> myVehicles = new ArrayList<>();
	myVehicles.add(first);
	myVehicles.add(second);
	myVehicles.add(third);

	for(Vehicle vehicle: myVehicles) {
		System.out.println(vehicle.getInteriorWidth());
	}

	}

}
```

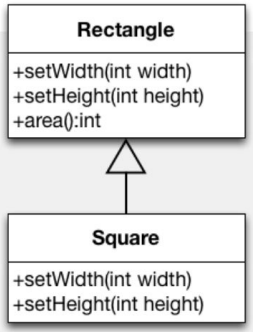
Violiting LSP

```
class Rectangle {
  public void setWidth(int width) {
    this.width = width;
  }
  public void setHeight(int height) {
    this.height = height;
  }
  public void area() {return height * width;}
  …
}
```

Square is a rectangle (mathematically speaking), let's implement ***Square*** as a subclass of ***Rectangle***.

Implementing `Square` as a subclass of `Rectangle`:

```
class Square extends Rectangle {
  public void setWidth(int width) {
        super.setWidth(width);
        super.setHeight(width);
  }
  public void setHeight(int height) {
        super.setWidth(height);
        super.setHeight(height);
  }
  …
}
```

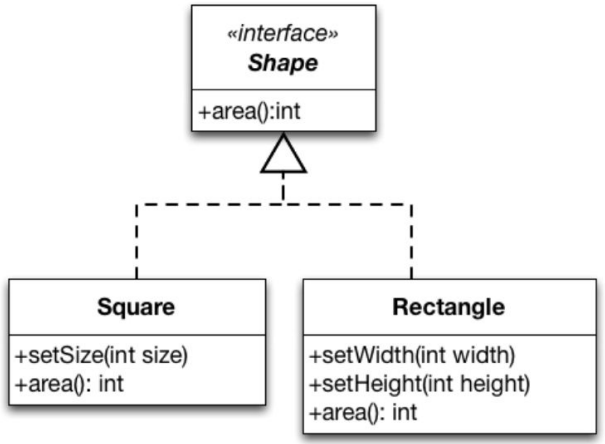


Solution

```
void clientMethod(Rectangle rec) {
  rec.setWidth(5);
  rec.setHeight(4);
  assert(rec.area() == 20);
}
```

- Introduce the interface shape
- To encapsulate what varies and to provide a generic interface we introduce an abstract shape class.

Rectangles and Square - LSP Compliant Solution



Advantages and Disadvantages LSP

Benefit

- Code that adheres to LSP is loosely dependent to each other and encourages code reusability.

Disadvantages

- Code that does not adhere to the LSP is tightly coupled and creates unnecessary entanglements.
    - E.g. when a subclass can not substitute its parent class there would have to be multiple conditional statements to determine the class or type to handle certain cases differently.

Interface Segregation Principle

No client should be forced to depends on method it don’t use

```java
public interface IMultiFunction {

    public void print();

    public void getPrintSpoolDetails();

    public void scan();

    public void scanPhoto();

    public void fax();

    public void internetFax();

}
```

```java
public class XeroxWorkCentre implements IMultiFunction
{

    @Override
    public void print() {
    // Assume real printing code
    }

    @Override
    public void getPrintSpoolDetails() {
    // Assume real code that gets spool details
    }

    @Override
    public void scan() {
    // Assume real code for scanning
    }

    @Override
    public void scanPhoto() {
    // Assume real code for photos scans
    }

    @Override
    public void fax() {
    // Assume real code for fax
    }

    @Override
    public void internetFax() {
    // Assume real code for internet fax
    }

}
```

Contd.

- Unimplemented methods are almost always indicative of poor designs
- Goes against ISP principle

HP PrinterNScanner

```
public class HPPrinterNScanner implements IMultiFunction
{

        @Override
        public void print() {
        // Assume real printing code
        }

        @Override
        public void getPrintSpoolDetails() {
        // Assume real code that gets spool details
        }

        @Override
        public void scan() {
        // Assume real code for scanning
        }

        @Override
        public void scanPhoto() {
        // Assume real code for photos scans
        }

        @Override
        public void fax() {

        }

        @Override
        public void internetFax() {

        }

}
```

Restructuring the Code to follow ISP

```
public interface IMultiFunction {

        public void print();

        public void getPrintSpoolDetails();

        public void scan();

        public void scanPhoto();

        public void fax();

        public void internetFax();

}
```

```
public interface IPrint {

        public void print();

        public void getPrintSpoolDetails();

}
```

```
public interface IScan {

        public void scan();

        public void scanPhoto();

}
```

```
public interface IFax {

        public void fax();

        public void internetFax();

}
```

Technique to find violation of ISP

Fat Interface - Fat interfaces mean interfaces with high number of methods.

Low Cohesion - Methods having entirely different function such Iscan , Ifax

Empty method Implementations - Blank implementations of interface methods is almost always a violation of the interface segregation principle.

SOLID principles complement each other, and work together in unison to achieve the common purpose of well-designed software.

Thank you